

Project Report in Advanced Object-Oriented Programming

Duc Huy Vu, Haron Obaid & Hieu Tran
Gr21

May 2023



1 Abstract

To learn the principles of Advanced Object-Oriented Programming. This project is about implementing the puzzle game Sokoban in Java. The game can be structured in many different ways, and it is up to the group to decide which design patterns to implement based on learned principles. The game can be played by navigating the player to assort crates according to the map design, with the course instructors providing icons. Through the implementation of the game implementing, specific types of patterns are emphasized. Focusing on modularity, reusability, and implementing the principled patterns of MVC, Subject-Observer, or Strategy. This Rapport will also go over the Design Patterns used, the issues the group had, and experiance with Version Contorll during the projects run.

Contents

1	Abstract	2
2	Introduction	4
2.1	Sokoban	4
2.2	Problems Encounterd	4
3	Design part	5
3.1	Model	6
3.1.1	BlockManager	6
3.1.2	Game	6
3.1.3	MapDesign	6
3.1.4	Level	6
3.1.5	LevelLoader	7
3.2	View	8
3.2.1	GUIView	8
3.2.2	TerminalView	8
3.2.3	Observer	8
3.3	Controller	9
3.3.1	ControllerTemplate & Direction	9
3.3.2	KeyboardHandler	9
3.3.3	MouseHandler	9
4	Testing & Debugging	10
5	Interesting parts	11
6	Results	13
7	Version Control/Git	14
8	Refrences	15

2 Introduction

The following project is made to understand project design better, improve the use of Java, and get more familiar with Advanced Object-Oriented Programming(AOOP) principles. The project aims to create a framework with a patterned design and to learn and get more comfortable working with different patterns. This project uses Model View Controller(MVC)[1], abstract templates, and observer patterns. Completing the project and this rapport equals 4.5 HP in the course Advanced Object-Oriented Programming.

2.1 Sokoban

As Wikipedia describes, Sokoban (Sōko-ban, lit. 'warehouse keeper') is a puzzle video game in which the player pushes boxes around in a warehouse, trying to get them to storage locations. The game was designed in 1981 by Hiroyuki Imabayashi and first published in December 1982[4]. The game is recognized as useful for developing problem-solving skills since the player needs to manage space in different capacities; the smallest errors might make the level impossible to complete. Only being able to push the crates also adds another degree of complexity. Sites such as mathisfun.com have implemented it as one of the games in their library. Sokoban is a 2D grid-based game focusing mainly on pushing crates in different directions. You win the specific level by setting all crates into the targeted positions.

2.2 Problems Encountered

There were problems during the implementation, such as moving the player accurately and collision. The issues were later resolved by checking two rows or columns forward from the player in the intended direction the player wants to move and determining if a collision will occur or if an object is in front of the player.

The group consisted of three people, and didn't all have the same vision for the project. Missing that you should start with the project in the 3rd week of the course start, and coming a bit late into the fold compared to the other groups. As this was the case, some group members focused on the third Lab of the course while the rest focused on the project. This led to the group not having a concrete plan when starting to code and essentially developing three game versions early on in the project's start. Some members working remotely further complicated matters, as coordinating the project wasn't always easy and impacted the code's modularity. This eventually got solved after reorganizing the project and meeting up about how we wanted to build the project further. We also encountered some problems when following the instructions for the MVC pattern and getting the logistics down for what should be accessed and should not be. Another pattern we had some issues implementing was the Subject-Observer pattern. Since the pattern relies on being updated in a specific way, we were unfamiliar with implementing such patterns. We gained further light on both patterns when the course instructors showed us examples after we presented the project for feedback.

3 Design part

The Program is split into three folders: Models, Controller, and View. Each folder represents an essential structure of the Program. The main pattern of the game starts with the MVC, which makes it easier to share the workload. While designing the Controller for the game, the implemented pattern is a Template method to make it easier to implement different pluggable methods for controlling the game. The observer pattern was later implemented to facilitate switching between a terminal view and a graphical view or a mixture of both. Since we use the Swing program like JPanel, the graphical view always needs to run as the Program closes without it. Some methods to circumvent this could be to implement another view strictly for the Terminal View using a Scanner to get the inputs. However, this might rely on some game loops that were forbidden to use in the project. The Program is called from the Master class Sokoban.java. The classes that are called are described below.



Figure 1: Block Diagram of the Program

3.1 Model

In the folder, Models contains classes that are a part of the Model part of the MVC pattern. These classes build the structure of the game. Its job is to manage how the player is supposed to move, the physics of the game, how the map is supposed to be built, and how the map updates and resets. Underneath are the Models classed in their alphabetical ordering.

3.1.1 BlockManager

The class BlockManager ensures that all the icons are uploaded properly and declares them as variables within the class. The class contains two matrices: one BufferedImage and one char. The BufferedImage matrix manages the game's states from a GUI's perspective, while the Program uses the char matrix to track the game's current state. By utilizing these matrices, the system can take less consideration to pixels management and instead focuses on updating the current state of the matrices. The char matrix also simplifies the implementation of the player icon into the game, as each character represents a block, making it easier to incorporate the game's logic when the player icon moves, push a block, or collides with a wall. Additionally, this approach facilitates the images used in the project. Even though BlockManager is part of the Model part of the project, it is still called underneath the graphical view part of the code GUIView. This is where it is better utilized, as exporting a char matrix is more accessible and less costly on the Memory than a BufferedImage matrix.

3.1.2 Game

Is the master Model class in charge of structuring the project and initializing its objects. It is also in charge of action events like going to the next level and calls for resets when the according input is reached. This is also where the observer classes are attached, and the different views can be implemented.

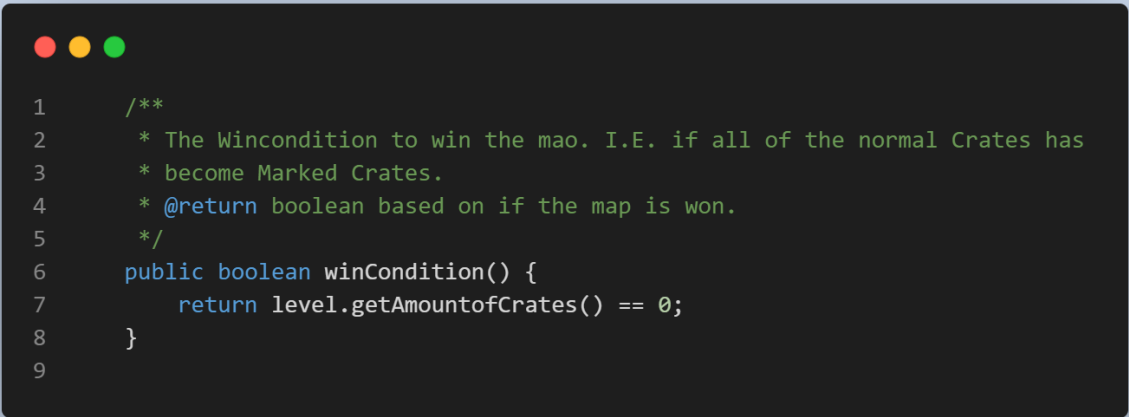
3.1.3 MapDesign

The class MapDesign designs maps that are used for the game. The class takes block variables from the class BlockManager to build the map. The class also contains methods for the game's core elements, such as moving the player, a collision between the player and objects, and pushing crates. To accomplish that, the class updates the current char matrix by checking the possible outcomes when a player moves and updates the map accordingly.

3.1.4 Level

The class Level holds the map's original state and will load it when called. It checks the current state of the map for the game to update properly. It returns the player's current position and number of crates, marked crates, and available targets. These methods are essential for it to make it easier to set up the win condition of the game, as well as make it easier to debug the game.

The game's win condition is established by simply having the Program check if any crates remain because the map has to be cleared with regular crates to win the game. When a regular crate is pushed onto a target, the game registers it as a marked crate instead, decreasing the number of crates.



```
1  /**
2   * The Wincondition to win the mao. I.E. if all of the normal Crates has
3   * become Marked Crates.
4   * @return boolean based on if the map is won.
5   */
6  public boolean winCondition() {
7      return level.getAmountofCrates() == 0;
8  }
9
```

Figure 2: the winning condition

3.1.5 LevelLoader

The class LevelLoader reads text files from the folder Levels and turns them into maps. The class can also check whether there are more levels in the folder levels, making it easier to implement as many levels as needed. The LevelLoader gives each Level a name based on the name of the file it was read from. It functions on these Level names and only reads the level when necessary making it very efficient not to use too much hardware memory to function as a program. LevelLoader is where the next level will be loaded by reading the level when called for if it exists.

3.2 View

The folder View contains all the classes responsible for showing the game's current state to the user. This includes both the console view and frame view. Different Views can easily be implemented through the Observer pattern, where every view is an observer and implements the observer interface. Below are the different Views implemented in their alphabetical order.

3.2.1 GUIView

GUIView is the graphical interface view that is implemented. Due to its relation to the Controller and the Java framework swing, it must be enabled. This graphical view is the model class BlockManager used, where the graphical images used in the project are used. The class GUIView receives updates from other classes to draw the map on the screen. For the class to update correctly, it uses the observer strategy by implementing the interface observer.

3.2.2 TerminalView

The class TerminalView displays the current state of the game in the console. The class implements the interface Observer to update the game's current state. It's a terminal-based view showing each new state like a log for the user. It could also be used to play the game if any graphical update error occurs. All of this makes it very useful for debugging purposes.

3.2.3 Observer

The observer is an interface that contains the method 'update.' The method has an argument input of a char matrix, and the purpose of the interface is to simplify the implementation of the observer strategy to the game. By including the method in the interface, the students can decide whether the terminal view or the GUI view to be running by simply implementing the method into the game.

3.3 Controller

The folder Controller consists of classes that plug in the keyboard and mouse input to the game. Handling keyboard and mouse events is essential to creating interactive user applications. We can use the KeyListener and MouseListener interfaces and the KeyEvent and MouseEvent classes to achieve this.

3.3.1 ControllerTemplate & Direction

The "ControllerTemplate" class has two main properties. The first is the private final GUIView window. This property is a reference to a GUIView object. It is passed in the constructor and used to interact with the GUI and update.

The second one is private Direction direction. It is the auditory attribute that represents the direction of the controller. The class Direction is an enum because the player can only be moved in four directions up, down, left, and right.

The "public ControllerTemplate (GUIView view)" function is the built-in way to enable it. This constructor takes a GUIView object as a parameter and assigns it to the window property, and at the same time, it also initializes the direction property to REST.

In this abstract class, we have a small method called public Direction getDirection(). This abstract method should be implemented by subclasses of "ControllerTemplate." It returns the value of the direction property.

The "ControllerTemplate" class follows the Template Method design pattern, where abstract methods are defined, and subclasses are responsible for implementing them. This class allows subclasses to customize and extend the controller's functionality. This class cannot be instantiated directly as it is abstract. It should be used as a template for their subclasses key and mouse.

3.3.2 KeyboardHandler

The KeyboardHandler class is a subclass of the ControllerTemplate class and implements the KeyListener interface. It processes and recognizes keyboard events to determine the player's movement direction and updates them immediately.

Featured:

1. Allows registration of arrow keys (UP, DOWN, RIGHT, LEFT) or corresponding keys (W, S, D, A) to indicate the direction of movement.
2. Support the SPACE key to reset the movement direction to the initial state (RESET).
3. Finally, Call the keyboardRegistration() method in the view object to update the UI after handling keyboard events.

3.3.3 MouseHandler

Like Keyhandler, The MouseHandler class is also a subclass of the ControllerTemplate class and implements the MouseListener interface. But the way to update their orientation is very different from the key handler. It processes mouse events and calculates to determine the direction of the player's movement based on the mouse position and the player's current position on the interface instead.

Featured:

1. Subscribe to the mouseClicked event to determine the mouse position and calculate the direction to move based on the distance between the mouse position and the player's current position.
2. Support movement direction: UP, DOWN, LEFT, RIGHT
3. Call the mouseRegistration() method in the view object to update the UI after handling mouse events.

Initializing the key handler and mouse handler will be set to REST by default. Now we have two complete key and mouse systems. To connect them to the GUIView, we will use the addKeyListener and addMouseListener code commands. Besides, we also must change this.setFocusable() to true so that the system can focus on listening to Controller and updating the information we have added from the key and mouse to update the view.

4 Testing & Debugging

The code was tested through the terminal view with several steps. One of the first tests was to see if the framework was set up properly and the map was printed out on the GUI. The test was done by experimenting with the columns and rows of the buffered matrix to see if each block was being placed properly. To set up a proper map, following the map design already created by the game Sokoban.

To first see if the core principles of the game are working, such as moving the player around and pushing the crate, the char map will be displayed in the terminal each time an object is moved. This made it easier to implement the game's logic, such as unmovable walls and the outcomes of a crate when it is pushed in different scenarios.

One of the ways to test whether the game was running correctly was to print out the player positions, the amounts of crates, marked crates, or targets on the map in the terminal. Each time the crate is pushed into a target, the value of the marked crate increases by one, and the amount of crates and targets decreases by one. If the terminal shows a different output than intended, it means either the GUI map is not updating correctly or there are errors in the code that need to be looked at.

However, there was an issue when implementing the method to change the value for the amounts of crates, marked crates, or targets. Every time the GUI map was updated, the value of these variables also increased on their own. It means that the value for amounts of targets, crates, and marked crates was not set but increased every time the map updated, making it inconsistent. The issue was solved by going through the char map instead. In this approach, the whole map doesn't update regularly, but moveable objects update their positions, and the quantities of crates and marked crates change themselves when a crate is pushed onto a target or when a marked crate is pushed outside a target.

A form of debugging used to a great extent was debugging using the breakpoint method. Since it was emphasized in previous courses, such as Computer System Engineering I using C, you don't always have an easy printing method. Using the breakpoint and Livewatch could be a very effective method to gather corresponding data to correct the bug in the program.

5 Interesting parts

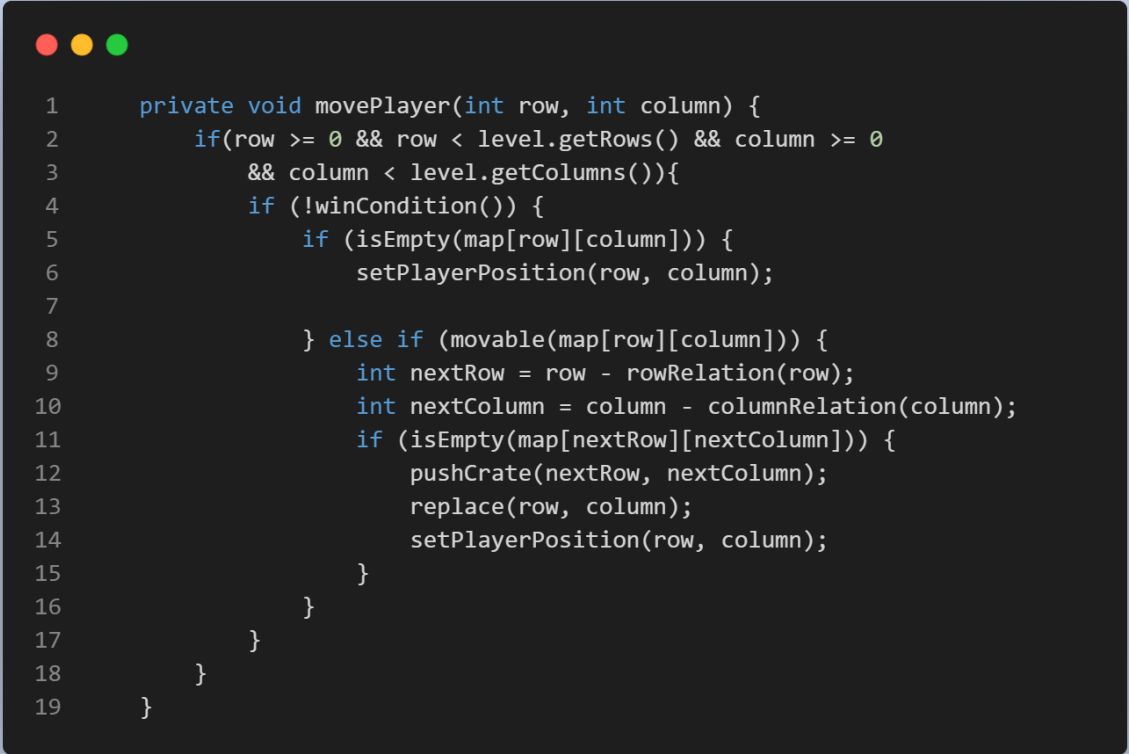
The logic behind updating moveable objects is complex, difficult, and interesting to implement. When an object or the player is moved, the row or column surrounding the object or player will be updated.

One scenario worth mentioning is when a player moves around the map or pushes a crate onto a target. After every movement from the player, the previous position will be replaced with a blank space to create the perception of continuous fluid movement effectively. Instead, the player is merely being placed upon another coordinate. When a crate is pushed into a target, the crate will transform into a marked crate. The primary logic focus on the importance of the 'replace' method to smoothly run the game.

```
1  /**
2   * Substitutes position that was used in action
3   * @param row to be replaced
4   * @param col to be replaced
5   */
6  private void replace(int row, int col) {
7      switch (map[row][col]) {
8          case '@' -> {
9              map[row][col] = '_';
10             }
11          case 'A' -> {
12              map[row][col] = 'o';
13             }
14          case 'C' -> {
15              map[row][col] = '_';
16             }
17          case 'M' -> {
18              map[row][col] = 'o';
19             }
20          case '_' -> {
21              map[row][col] = '_';
22             }
23          case 'o' -> {
24              map[row][col] = 'o';
25             }
26          default -> {}
27      }
28  }
29  }
```

Figure 3: What to reset for each unique scenario

Another essential component for the game to run smoothly is the 'movePlayer' method. The method examines the current positions of the player and predicts the possible outcome when the player moves. If an obstacle is blocking the player's path, the process determines if its moveable or not. The object is only moveable if it's a crate, not in front or behind a wall or another crate. The method 'move player' is initiated when a player movement occurs, resulting from when a key button is pressed or the mouse is clicked on the map. The player will then move around with the movement logic. Every time the methods 'move player' and 'replace' are invoked, the char and GUI matrix update, providing a view for the user. The pattern MVC(Model View Controller) becomes evident throughout this process, making it one of the ideal patterns to implement in the game Sokoban.



```
1  private void movePlayer(int row, int column) {
2      if(row >= 0 && row < level.getRows() && column >= 0
3          && column < level.getColumns()){
4          if (!winCondition()) {
5              if (isEmpty(map[row][column])) {
6                  setPlayerPosition(row, column);
7
8              } else if (movable(map[row][column])) {
9                  int nextRow = row - rowRelation(row);
10                 int nextColumn = column - columnRelation(column);
11                 if (isEmpty(map[nextRow][nextColumn])) {
12                     pushCrate(nextRow, nextColumn);
13                     replace(row, column);
14                     setPlayerPosition(row, column);
15                 }
16             }
17         }
18     }
19 }
```

Figure 4: The logic of player movement

A third point of interest is the observation of the programmed scenarios. For simpler code, you don't need to program scenarios that will never happen. I.E., a player moving over a wall or if the wall is replaced. Therefore there is no need to look if the wall needs to be moved, repainted, or replaced. Making our switch cases a bit less complex and more manageable.

6 Results

The project is a success. It manages to implement both an MVC pattern and a Subject Observer pattern. These patterns are essential to making the program modular and friendly for future features. As it follows both patterns, most programmers with a background in Object-Oriented Programming will easily understand the program's structure.

It does work as intended, and you can add as many levels as you have available space. You can also implement new views, such as a terminal-only view or a good one. This structure also makes it friendly to reuse process frameworks for future projects.

Some features like a menubar to reset or navigate the Levels, another view, for example, the sound could also be wanted. Both are and will be easy to implement with a bit more time and effort, and the framework will not need any major reworks from the current state. Hence the project is a success.

7 Version Control/Git

Version Control or Git was more accessible for some members and harder for others. If you don't regularly spend time with the program, you often forget ways of implementing, saving, and merging code segments. Another level of complexity was the usage of different Integrated Development Environments, or IDEs for short, as other programs set up the Java project in different ways. Finding a common form of implementing the code to the git repository might sometimes cause headaches. Using SSH keys was valuable for the smoothness of merging the codes. As the keys automatically verify the user to the storage, it was a simple and very effective method of merging. Another highlight would be the Visual Studio Codes Push & Pull method. This integrated way of merging code to the git repository on a specific website makes the experience way more user-friendly, as the user barely needs to remember any command in the terminal after the installation process if they don't want to. As the project progressed, all the members became more familiar with at least the project's main version control methods.

8 References

References

- [1] Codecademy. Mvc: Model, view, controller. <https://www.codecademy.com/article/mvc>, 2023.
- [2] Refactoring Guru. Observer. <https://refactoring.guru/design-patterns/observer>, 2023.
- [3] Refactoring Guru. Strategy. <https://refactoring.guru/design-patterns/strategy>, 2023.
- [4] Wikipedia. Sokoban. <https://en.wikipedia.org/wiki/Sokoban>, 2023.